

Prometheus Cheat Sheet

Current 3.x line (latest 3.14.0). Quick reference only — verify against the official docs before production use.

METRIC TYPES

Type	Shape / use
Counter	monotonic total; use <code>rate()</code> , never raw value
Gauge	value that can go up/down (temp, queue size)
Histogram	<code>_bucket{le} + _sum + _count</code> ; server-side quantiles
Summary	client-side quantiles + <code>_sum/_count</code> ; not aggregatable

Native histograms (exponential buckets) trade exact `le` boundaries for far lower cardinality.

PROMQL SELECTORS

```
http_requests_total{job="api",code=~"5.."}
http_requests_total[5m]           # range vector
http_requests_total[5m:1m]        # subquery
http_requests_total offset 1h
rate(http_requests_total[5m] @ 1609459200)
```

Matchers: `=` `!=` `=~` `!~` (RE2, anchored).

PROMQL OPERATORS

- Arithmetic `+` `-` `*` `/` `%` `^`, comparison `==` `!=` `>` `<` `>=` `<=`, logical `and` `or` `unless`
- Vector matching: `on(...)`, `ignoring(...)`, `group_left/group_right` for many-to-one
- Aggregation: `sum by(...)` (`x`), `avg`, `max`, `min`, `count`, `topk(n, x)`

PROMQL FUNCTIONS

- `rate(c[5m]) / irate(c[5m])` — per-second average / instantaneous
- `increase(c[5m])` — extrapolated total increase over the range
- `histogram_quantile(0.95, sum by(le) (rate(h_bucket[5m])))`
- `predict_linear(g[1h], 4*3600)` — linear forecast
- `label_replace(v, "dst", "$1", "src", "(.*)")`, `label_join(...)`

SCRAPE_CONFIGS SKELETON

```
scrape_configs:
- job_name: node
  scrape_interval: 15s
  static_configs:
  - targets: ["host:9100"]
  relabel_configs:
  - source_labels: [__address__]
    target_label: instance
- job_name: k8s-pods
  kubernetes_sd_configs:
  - role: pod
```

RELABELING

`relabel_configs` runs before scraping (can drop/rewrite the target itself); `metric_relabel_configs` runs after, per scraped sample.

```
action: keep|drop|replace|labelmap|
       labeldrop|hashmod
```

`honor_labels: true` lets scraped labels win over target labels on collision.

ALERTING RULE SKELETON

```
groups:
- name: example
  rules:
  - alert: HighErrorRate
    expr: |
      sum(rate(http_requests_total{code=~"5.."}
[5m]))
      / sum(rate(http_requests_total[5m])) > 0.05
    for: 10m
    labels: {severity: page}
    annotations:
      summary: "Error rate above 5%"
```

RECORDING RULE

```
- record: job:http_requests:rate5m
  expr: sum by(job)(rate(http_requests_total[5m]))
```

Precompute expensive/frequent queries; name as `level:metric:operations`.

PROMTOOL ONE-LINERS

```
promtool check config prometheus.yml
promtool check rules alerts.yml
promtool test rules rules_test.yml
promtool query instant http://host:9090 'up'
promtool query range http://host:9090 \
'rate(http_requests_total[5m])' \
--start=2024-01-01T00:00:00Z --end=... --step=1m
```

HTTP API & CURL

```
curl -g 'http://host:9090/api/v1/query?query=up'
curl 'http://host:9090/api/v1/query_range?
query=up&start=...&end=...&step=15s'
curl -X POST http://host:9090/-/reload
curl http://host:9090/api/v1/rules
curl http://host:9093/api/v2/silences
```

ALERTMANAGER ROUTING

Route tree matches on labels (`match/match_re`) top-down; `group_by/group_wait/group_interval/repeat_interval` control batching and repeats.

Inhibition suppresses one alert while another (matching `source_match`) is firing; silences mute by label matcher for a time window.

SERVICE DISCOVERY

- `static_configs`, `file_sd_configs`, `http_sd_configs`
- `kubernetes_sd_configs` roles: `node`, `pod`, `service`, `endpoints`, `ingress`
- `consul_sd_configs`, `ec2_sd_configs`

Discovered targets always pass through `relabel_configs` before being scraped.

EXPOSITION FORMAT

```
# HELP http_requests_total Total requests
# TYPE http_requests_total counter
http_requests_total{method="GET",code="200"} 1027
```

Plain text or OpenMetrics (Accept: `application/openmetrics-text`); client libraries handle label escaping and the trailing `# EOF`.

GRAFANA QUICK REFERENCE

- Add Prometheus as a data source (URL + scrape/query timeout)
- Panel types: time series, stat, gauge, table, heatmap
- Template variables: `label_values(metric, label)`, used as `$var` in queries
- Grafana-managed alerting vs. Prometheus/Alertmanager alerting; notification policies & contact points